First, we must remember to retarget our swarm:

```
$ eval $(docker-machine env local-vm-1)
```

Now we can deploy our app with the command:

```
$ docker stack deploy -c docker-stack.yml myapp
Creating network myapp_default
Creating service myapp_web
Creating service myapp_redis
Creating service myapp_database
Creating service myapp_db-migrator
```

This says, "Deploy the services described in docker-stack.yml as a stack called myapp." You can see the various services being created.

We can list the services in our stack by running:

```
$ docker stack services myapp
ID   NAME                MODE  REP…  IMAGE                  PORTS
p9…  myapp_db-migrator   rep…  0/1   robisenberg/myapp_w…
s5…  myapp_web           rep…  1/1   robisenberg/myapp_w…  *:80->3000/tcp
ue…  myapp_database      rep…  1/1   postgres:latest
ws…  myapp_redis         rep…  1/1   redis:latest
```

Services deployed on Swarm can be scaled up by creating additional *replica* containers for the service. The "replicas" column shows how many replicas are running vs. the desired number of replicas (which, by default, is one).

When you run this command, you'll probably find that one or more of our services haven't started up yet, indicated by "0/1" in the "replicas" column. It may take a while until all the services have started, especially on the very first deploy, since each service's image needs to be downloaded from scratch. Wait until all the services except db-migrator have started by rerunning the command and waiting until the "replicas" column shows "1/1" (db-migrator runs and then terminates).

As a shortcut, you can also run the similar command:

```
$ docker service ls
ID   NAME                MODE  REP…  IMAGE                  PORTS
ue…  myapp_database      rep…  1/1   postgres:latest
p9…  myapp_db-migrator   rep…  0/1   robisenberg/myapp_w…
ws…  myapp_redis         rep…  1/1   redis:latest
s5…  myapp_web           rep…  1/1   robisenberg/myapp_w…  *:80->3000/tcp
```

This lists all the services that have been deployed to the swarm. With multiple stacks deployed on the swarm, you'd see the services for all stacks listed here. However, since we only have a single stack deployed, the output is the same as for docker stack services myapp, which lists the services for a *specific* stack.